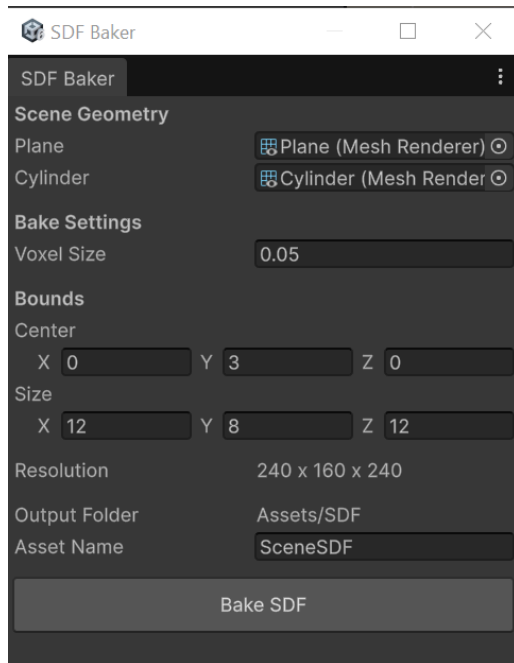


Unity 6.3 LTS (6000.3.24f1) URP

I created a custom Unity Editor window for generating the SDF.

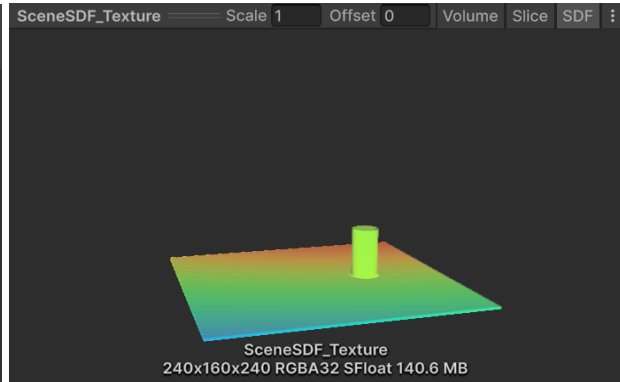


How to use:

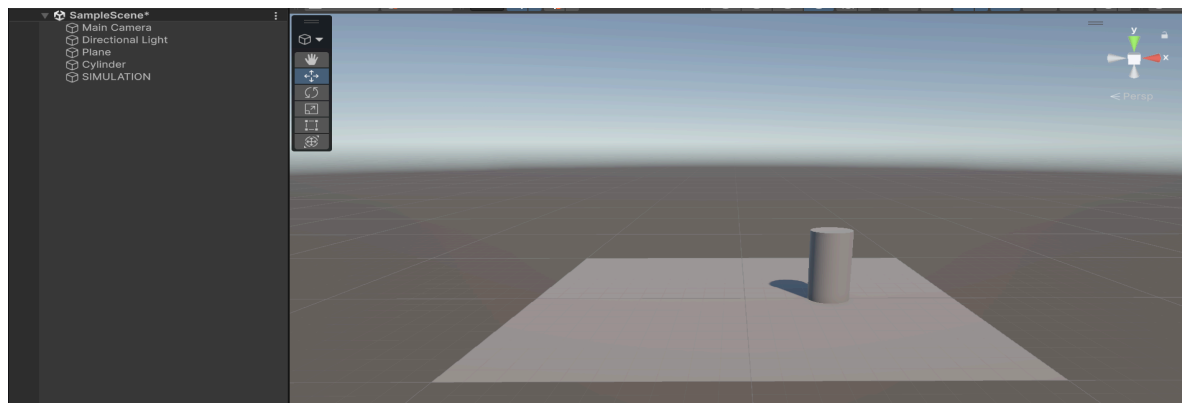
1. in the topbar go to **Tools** → **SDF** → **Baker**.
2. Assign the scene's **Plane** and **Cylinder** to the corresponding fields. You can move them freely in the scene.
3. set the desired **Voxel Size**.
4. set the SDF **Bounds Center** and **Bounds Size**.
5. specify the output asset name.
6. click "Bake SDF" to generate files.
7. assign the generated **SDFDataSO** to the **SDF Volume** component on the **SIMULATION GameObject**.
8. enter Play Mode.

It will show you the resolution of Texture3D and the output folder.

Output Example:



Scene Example:



I have a Dell XPS 13 with integrated graphics (no discrete GPU). 10000 balls is close to the limit on this hardware. In a build with all other applications closed I reach ~55 FPS, and around 40 FPS in the Editor. To separate hardware limitations from implementation scaling, I also tested the standalone build on several discrete GPUs.

For all tests I used:

- Voxel Size = 0.05
- Bounds Center = (0, 3, 0)
- Bounds Size = (12, 8, 12)
- SubSteps = 4
- Radius = 0.05
- Low-poly ball mesh. I used an icosphere created in Blender to keep the vertex count low.

Results for my device inside Unity Editor:

Intel(R) Iris(R) Xe Graphics

Balls	FPS	Sim (ms)	CPU (ms)	GPU (ms)
10 000	~38	~0.86	~26 ms	~26 ms
20 000	~24	~1.82	~41 ms	~41 ms
40 000	~14	~6.07	~71 ms	~71 ms
80 000	~7	~8.15	~138 ms	~138 ms
160 000	~4	~16.88	~261 ms	~261 ms

Results inside Build (I asked my friends to test it on their hardware):

RED - Intel(R) Iris(R) Xe Graphics

YELLOW - NVIDIA GeForce GTX 1050 Ti

GREEN - NVIDIA GeForce GTX 1650 Ti

BLUE - NVIDIA GeForce RTX 5060 Ti

Balls	FPS	Sim(ms)		FPS	Sim(ms)		FPS	Sim(ms)		FPS	Sim(ms)
10k	~55	~1.76		~74	~1.35		~100	~1.98		~700	~0.2
20k	~38	~2.62		~61	~2.01		~83	~3.42		~374	~0.35
40k	~23	~7.60		~31	~4.06		~70	~3.72		~257	~0.82
80k	~14	~12.30		~17	~7.22		~43	~6.15		~142	~1.55
160k	~8	~19.81		~9	~25.28		~28	~23.56		~61	~3.37

Where does the implementation fall over and why?

10,000 balls is comfortable on all tested hardware. Performance starts to degrade significantly around 40,000 – 80,000 balls, depending on the hardware.

The simulation cost scales with the number of balls and substeps, as expected. For example, on my system the simulation time increases from 1.76 ms at 10k balls to 19.81 ms at 160k balls.

However, simulation is not the only cost. At 160k balls, for example, the RTX 5060 Ti test shows approximately 3.37 ms of simulation time but only ~61 FPS (~16.4 ms frame time). This indicates that rendering and the CPU-to-GPU part of the pipeline become significant at higher instance counts.

The main factors I would expect to limit further scaling are:

- SDF sampling and collision work for every ball and substep.
- Synchronization with `JobHandle.Complete()` before the results can be uploaded.
- Uploading the position buffer to the GPU every frame (~2.56 MB/frame at 160k balls).
- Processing and rendering a growing number of mesh instances on the GPU.

The exact contribution of each part would require more detailed profiling. The tests on different GPUs show that the practical limit is strongly hardware-dependent.